

[Sign in](#)[VasaviDappili](#) / [Python-Function-Assignments](#) Public[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#)[main](#)[Go to file](#)[Code](#) **VasaviDappili** Update README.md

6ba954b · 39 minutes ago

README.md

Update README.md

39 minutes ago

README



Python-Function-Assignments

1. Write a function `calculate(a, b, operation)` that performs addition, subtraction, multiplication, or division based on the supplied operation.

Code

```
def calculate(a, b, operation):
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b == 0:
            return "Division by zero is not allowed"
        return a / b
```



```
    else:
        return "Invalid operation"

print("Addition:", calculate(20, 10, "add"))
print("Subtraction:", calculate(20, 10, "subtract"))
print("Multiplication:", calculate(20, 10, "multiply"))
print("Division:", calculate(20, 10, "divide"))
```

Output

```
In [1]: def calculate(a, b, operation):
        if operation == "add":
            return a + b
        elif operation == "subtract":
            return a - b
        elif operation == "multiply":
            return a * b
        elif operation == "divide":
            if b == 0:
                return "Division by zero is not allowed"
            return a / b
        else:
            return "Invalid operation"

print("Addition:", calculate(20, 10, "add"))
print("Subtraction:", calculate(20, 10, "subtract"))
print("Multiplication:", calculate(20, 10, "multiply"))
print("Division:", calculate(20, 10, "divide"))

Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2.0
```

2. Write a function sum_numbers(*args) that accepts any number of arguments and returns their sum.

Code

```
def sum_numbers(*args):
    total = 0

    for value in args:
        total = total + value

    return total

print("Sum:", sum_numbers(10, 20, 30))
print("Sum:", sum_numbers(5, 10, 15, 20, 25))
print("Sum:", sum_numbers())
```



Output

```
In [2]: def sum_numbers(*args):
        total = 0

        for value in args:
            total = total + value

        return total

print("Sum:", sum_numbers(10, 20, 30))
print("Sum:", sum_numbers(5, 10, 15, 20, 25))
print("Sum:", sum_numbers())

Sum: 60
Sum: 75
Sum: 0
```

3. Write a function `employee(** args)` that accepts employee information such as name, ID, department and salary, then displays the information.

Code

```
def employee(**args):
    print("Employee Details")

    for key, value in args.items():
        print(key, ":", value)

employee(
    name="Ravi",
    ID=101,
    department="IT",
    salary=35000
)
```



Output

```
In [3]: def employee(**args):
        print("Employee Details")

        for key, value in args.items():
            print(key, ":", value)

employee(
    name="Ravi",
    -- --
```

```
ID=101,  
department="IT",  
salary=35000  
)
```

```
Employee Details  
name : Ravi  
ID : 101  
department : IT  
salary : 35000
```

4. Write a function `remove_duplicates(lst)` that returns a list containing only unique elements while preserving their original order.

Code

```
def remove_duplicates(lst):  
    result = []  
  
    for item in lst:  
        if item not in result:  
            result.append(item)  
  
    return result  
  
numbers = [4, 2, 4, 7, 2, 9, 7, 5]  
  
print("Original list:", numbers)  
print("After removing duplicates:", remove_duplicates(numbers))
```

Output

```
In [4]: def remove_duplicates(lst):  
        result = []  
  
        for item in lst:  
            if item not in result:  
                result.append(item)  
  
        return result  
  
numbers = [4, 2, 4, 7, 2, 9, 7, 5]  
  
print("Original list:", numbers)  
print("After removing duplicates:", remove_duplicates(numbers))  
  
Original list: [4, 2, 4, 7, 2, 9, 7, 5]  
After removing duplicates: [4, 2, 7, 9, 5]
```

5. Using a lambda function, sort a list of

tuples based on the second element.

Example: [(1,5), (2,3), (4,1)].

Code

```
data = [(1, 5), (2, 3), (4, 1)]

sorted_data = sorted(data, key=lambda item: item[1])

print("Original list:", data)
print("Sorted list:", sorted_data)
```



Output

```
In [5]: data = [(1, 5), (2, 3), (4, 1)]
sorted_data = sorted(data, key=lambda item: item[1])
print("Original list:", data)
print("Sorted list:", sorted_data)
Original list: [(1, 5), (2, 3), (4, 1)]
Sorted list: [(4, 1), (2, 3), (1, 5)]
```

Releases

No releases published

Contributors 1



VasaviDappili

[Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#)

Do not share my personal information



© 2026 GitHub, Inc.